

ESP32-Based Vehicle Stalling Simulator: A Low-Cost Embedded System for Driver-Training Applications

Mogana G
23BEC0172
Department of Electronics and
Communication Engineering
VIT University
Vellore, India
mogana.g2023@vitstudent.ac.in

Skandha Pavithra S
23BEC0103
Department of Electronics and
Communication Engineering
VIT University
Vellore, India
skandhapavithra.s2023@vitstudent.ac.in

Sreya Saji
23BVD0061
Department of Electronics and
Communication Engineering
VIT University
Vellore, India
sreya.saji2023@vitstudent.ac.in

Abstract

This paper presents the design, implementation, and simulation of an ESP32-based vehicle stalling simulator for driver-training and educational applications. The system uses three 10 k Ω potentiometers to emulate clutch position, engine speed (RPM), and vehicle speed, together with a momentary push button representing the brake pedal. Real-time feedback is provided through a 16 $\times$ 2 I²C LCD, a red warning LED, a green status LED, and an active buzzer. A rule-based stall-detection algorithm triggers alerts when clutch engagement exceeds 10% and either engine speed falls below 1,000 RPM or vehicle speed exceeds 5 km/h. An independent brake-overspeed warning activates when the brake is applied at speeds above 50 km/h. Hardware co-simulation is performed in the Wokwi environment. All five defined test cases passed, demonstrating the viability of the proposed architecture as a portable, affordable pedagogical tool.

Keywords— *ESP32; vehicle simulator; stall detection; embedded systems; driver training; IoT; real-time control.*

I. Introduction

Road traffic fatalities remain a significant public health concern, and driver inexperience as well as improper speed and gear management are widely cited contributing factors [1]. Manual-transmission vehicles are particularly unforgiving of clutch misuse: an inadvertent stall at an intersection or during an overtaking manoeuvre can lead to loss of propulsion and reduced vehicle controllability. Traditional driving simulators capable of reproducing these dynamics are often expensive, bulky, and less accessible to institutions in resource-constrained environments.

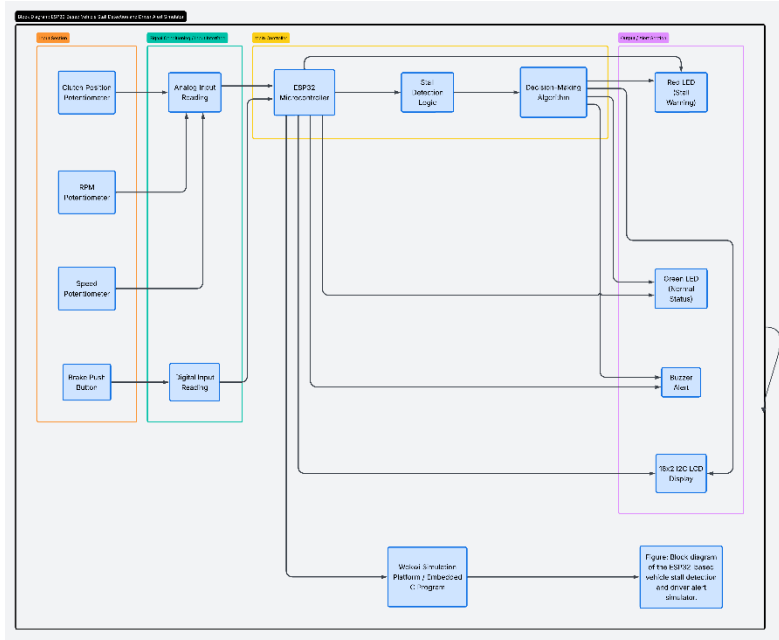
Low-cost microcontroller platforms, epitomised by the Espressif ESP32, have matured sufficiently to support real-time sensor fusion, analogue-to-digital conversion, and peripheral interfacing within a single system-on-chip. The present work exploits these capabilities to construct a compact hardware simulator that replicates the principal mechanical interlocks governing vehicle stalling. Analogue potentiometers provide continuously variable inputs equivalent to clutch position, engine RPM, and road speed, while digital I/O handles brake actuation and alert output.

The main objectives of this work are:

- (i) Design a rule-based stall detection engine suitable for execution on a 240 MHz dual-core microcontroller.
- (ii) Validate the system through structured simulation in the Wokwi platform.
- (iii) Demonstrate applicability as an affordable supplement to formal driver-training curricula.

II. System Architecture

Fig. 1 shows the top-level block diagram. The architecture comprises three functional layers: (1) the sensing layer, formed by three potentiometers and one push button; (2) the processing layer, implemented entirely on the ESP32 SoC; and (3) the alerting layer, consisting of the LCD, LEDs, and buzzer.



The sensing layer feeds 12-bit ADC readings into the processing layer at a 100 ms polling interval. Firmware maps raw ADC values to engineering units (% , RPM, km/h) using linear scaling, then evaluates two Boolean rules. Outputs are updated within the same loop iteration, yielding a typical end-to-end response time below 200 ms, which is sufficient for human-perceptible real-time feedback.

Software is structured as a single Arduino sketch with four logical blocks: (a) hardware initialisation, (b) sensor acquisition and scaling, (c) decision logic, and (d) output rendering. Modularity is maintained by encapsulating display updates and alert state transitions in dedicated helper functions.

III. Hardware Design

A. Component Selection

Table I lists all components with their specifications. The ESP32 DevKit v1 was selected for its dual 12-bit SAR ADC banks, hardware I²C peripheral, and abundant GPIO count. Potentiometers are wired in a simple voltage-divider configuration between 3.3 V and GND, placing the wiper output directly on ADC input-only pins (GPIO 34, 35, 32) to avoid unintended drive contention. The 16×2 LCD with PCF8574 I²C backpack reduces the wiring harness to two signal lines, SDA (GPIO 21) and SCL (GPIO 22), each pulled high by 4.7 kΩ resistors.

TABLE I — Component Specifications

Component	Model / Spec	Function
ESP32 MCU	38-pin DevKit v1	Core processing, ADC, GPIO
Potentiometers (×3)	10 kΩ linear taper	Clutch, RPM, Speed simulation
Push Button	Momentary NO, 6mm	Brake pedal input
LCD Display	16×2 I ² C (PCF8574)	Real-time parameter display
Active Buzzer	5V, 85 dB, continuous	Audible stall/brake alert
Red LED	5mm, 2.2V, 20mA	Warning indicator
Green LED	5mm, 2.0V, 20mA	Normal-operation indicator
Resistors (×2)	220 Ω, 0.25 W	LED current limiting

B. Pin Connections

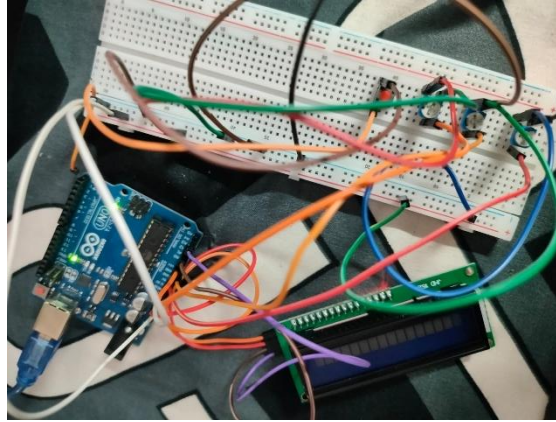
Table II summarises the GPIO mapping.

TABLE II — ESP32 GPIO Mapping

Signal	ESP32 Pin	Notes
Clutch POT	GPIO 34 (ADC1_CH6)	Input-only; 0–3.3 V analogue
RPM POT	GPIO 35 (ADC1_CH7)	Input-only; mapped 0–8000 RPM
Speed POT	GPIO 32 (ADC1_CH4)	Mapped 0–200 km/h
Brake Button	GPIO 27	Internal pull-up; active LOW
LCD SDA	GPIO 21	I ² C data; 4.7 k Ω pull-up
LCD SCL	GPIO 22	I ² C clock; 4.7 k Ω pull-up
Buzzer	GPIO 26	Active HIGH
Red LED	GPIO 25	220 Ω series resistor
Green LED	GPIO 33	220 Ω series resistor

C. Power Considerations

The system is powered via the ESP32 USB micro-B connector (5 V, ≥ 500 mA). The on-board AMS1117 linear regulator provides the 3.3 V rail consumed by the ESP32 core and the analogue sensing elements. The LCD module accepts 5 V directly and draws approximately 30 mA with the backlight enabled. With 220 Ω series resistors, the LED currents are approximately $(3.3 \text{ V} - 2.2 \text{ V}) / 220 \Omega \approx 5.0 \text{ mA}$ (red) and $(3.3 \text{ V} - 2.0 \text{ V}) / 220 \Omega \approx 5.9 \text{ mA}$ (green), which are within typical GPIO drive capabilities for indicator LEDs. Total system current consumption is estimated at 280 mA, leaving adequate headroom for a 500 mA USB host.



IV. Software Design

A. Logic Flow

Fig. 3 depicts the firmware flow chart. After one-time peripheral initialisation the main loop executes the following sequence every 100 ms: read ADC channels $\rightarrow$ scale to engineering units $\rightarrow$ evaluate stall and brake-warning predicates $\rightarrow$ update outputs $\rightarrow$ refresh LCD.

B. Key Algorithms

The stall predicate is expressed as a single Boolean condition (thresholds in % engagement, RPM, and km/h):

$$\text{STALL} \leftarrow (\text{clutch} > 10\%) \wedge ((\text{RPM} < 1,000) \vee (\text{speed} > 5))$$

The brake-overspeed predicate is evaluated independently so that a brake warning may co-exist with a SAFE clutch state:

$$\text{BRAKE_WARN} \leftarrow \text{brakePressed} \wedge (\text{speed} > 50)$$

Both predicates are evaluated after every sensor-acquisition cycle. If both evaluate true simultaneously, the firmware displays the stall condition in preference to the brake warning, while still asserting the relevant outputs as configured.

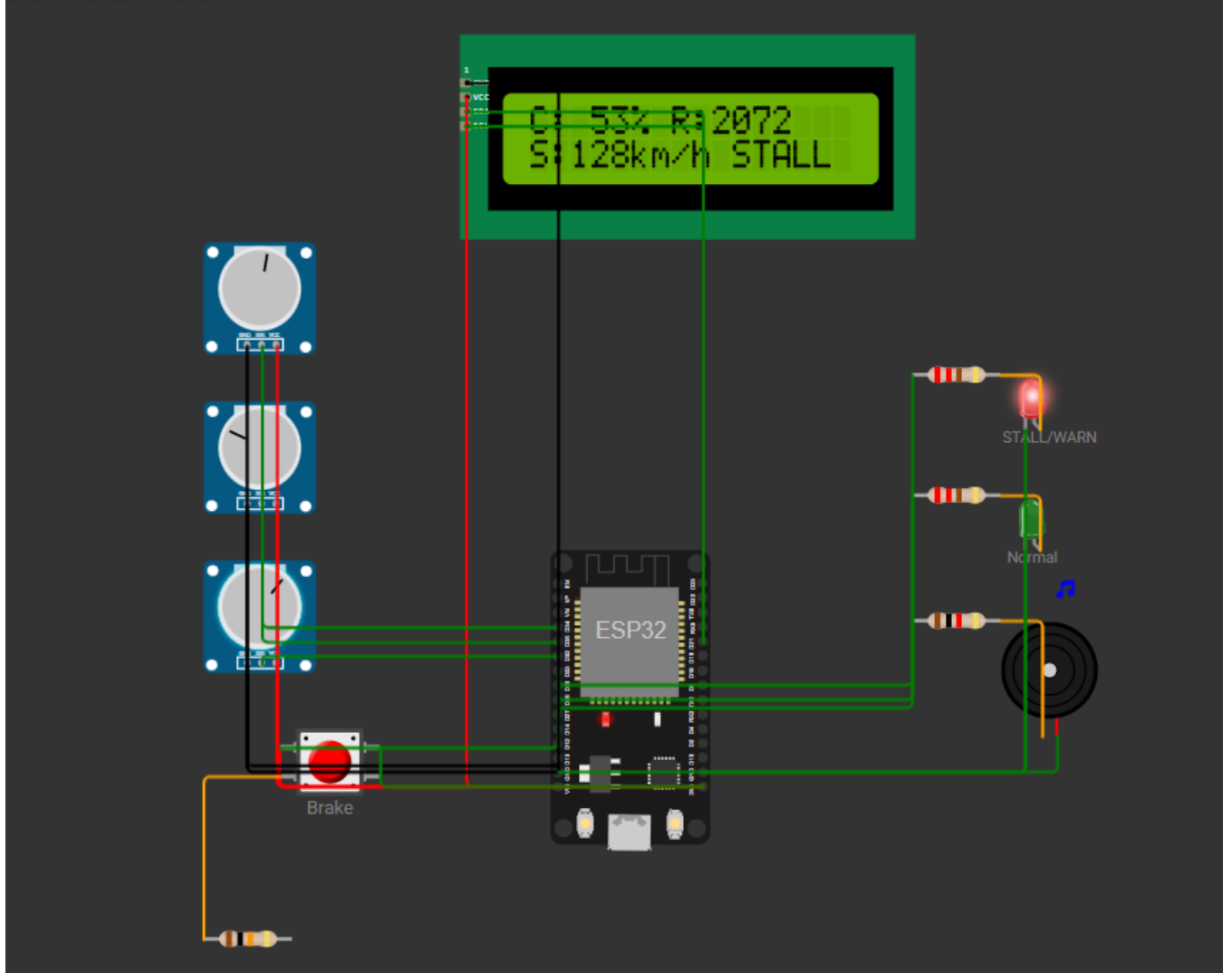
C. Code Structure Overview

The sketch is organised into six sections: library includes and pin definitions; global-state variables; `setup()` for peripheral initialisation and LCD splash screen; `readSensors()` for ADC acquisition and scaling; `evaluateLogic()` for predicate evaluation and output assertion; and `updateDisplay()` for I²C LCD rendering. Inline comments describe each operational block for maintainability.

V. Simulation and Testing

A. Wokwi Simulation Environment

Wokwi is a browser-based circuit simulator that supports the ESP32 Arduino core, the LiquidCrystal_I2C library, and interactive virtual components including potentiometers and push buttons. The simulation was configured using a `diagram.json` project file specifying component attributes, GPIO assignments, and net connections. Virtual serial output was monitored alongside the LCD state to verify scaling and decision-logic behaviour.



B. Test Cases and Results

Five structured test cases were defined to exercise all decision branches. Table III summarises the inputs, expected system state, and observed results. All tests passed.

TABLE III — Test Case Results

#	Clutch / RPM / Speed	Brake	Expected State	Result
1	5% / 2500 / 40	Off	SAFE – Green LED	✓ Pass
2	30% / 700 / 0	Off	STALL – Red + Buzz	✓ Pass
3	50% / 3000 / 80	Off	STALL – Red + Buzz	✓ Pass
4	5% / 2000 / 80	On	BRAKE! – Red + Buzz	✓ Pass
5	5% / 1500 / 30	On	SAFE (speed $\leq$ 50)	✓ Pass

Code:

```

* =====
* ESP32 Vehicle Stall Simulator
* Version: 1.0
* Target: ESP32 DevKit V1 (38-pin or 30-pin)
* =====
* HARDWARE CONNECTIONS:
* Potentiometers (10k $\Omega$  each):
*   Clutch POT → GPIO 34 (ADC1_CH6) — ADC-only pin, input only
*   RPM POT   → GPIO 35 (ADC1_CH7) — ADC-only pin, input only
*   Speed POT  → GPIO 32 (ADC1_CH4)
*   All POT outer legs: one to 3V3, other to GND
*   All POT wipers: to respective GPIO above
*
* Brake Button (momentary NO):
*   One leg → GPIO 33
*   Other leg → GND
*   10k $\Omega$  pull-up between GPIO33 and 3V3
*
* Outputs:
*   Red LED → 220 $\Omega$  resistor → GPIO 25
*   Green LED → 220 $\Omega$  resistor → GPIO 26
*   Buzzer (active, 5V, driven via NPN 2N2222):
*   GPIO 27 → 1k $\Omega$  → Base of 2N2222
*   Collector → Buzzer(+) → 3V3 (or 5V via Vin)
*   Emitter → GND
*   Buzzer(-) → GND
*   LCD 16 $\times$ 2 I2C (PCF8574 backpack):
*   SDA → GPIO 21
*   SCL → GPIO 22
*   VCC → 3V3 or 5V (check your module spec)
*   GND → GND
* =====
* LIBRARIES REQUIRED (install via Arduino Library Manager):

```

```

* - LiquidCrystal_I2C by Frank de Brabander (v1.1.2+)
* Built-in (no install needed):
* - Wire.h (I2C, bundled with ESP32 core)
* =====
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ——— Pin Definitions —————
#define PIN_POT_CLUTCH 34 // Analog: clutch position
#define PIN_POT_RPM 35 // Analog: engine RPM
#define PIN_POT_SPEED 32 // Analog: vehicle speed
#define PIN_BTN_BRAKE 33 // Digital input: brake pedal
#define PIN_LED_RED 25 // Digital output: warning LED
#define PIN_LED_GREEN 26 // Digital output: normal LED
#define PIN_BUZZER 27 // Digital output: active buzzer

// ——— Physical Range Mapping —————
// ADC on ESP32 is 12-bit → 0–4095
#define ADC_MAX 4095
#define CLUTCH_MIN 0.0f
#define CLUTCH_MAX 100.0f // percentage
#define RPM_MIN 0.0f
#define RPM_MAX 8000.0f // RPM
#define SPEED_MIN 0.0f
#define SPEED_MAX 200.0f // km/h

// ——— Stall / Warning Thresholds —————
#define STALL_CLUTCH_THRESHOLD 10.0f // % — stall starts above this
#define STALL_RPM_THRESHOLD 1000.0f // RPM — stall if below this
#define STALL_SPEED_THRESHOLD 5.0f // km/h — stall if above this
#define BRAKE_SPEED_THRESHOLD 50.0f // km/h — brake warning above this

// ——— Buzzer Timing —————
#define BUZZER_BEEP_ON_MS 150 // ms buzzer ON per cycle
#define BUZZER_BEEP_OFF_MS 150 // ms buzzer OFF per cycle (fast = stall)
#define BUZZER_BRAKE_ON_MS 80 // ms for brake warning (faster)
#define BUZZER_BRAKE_OFF_MS 80

// ——— LCD Setup —————

```

```
// Change address to 0x3F if 0x27 doesn't work (run I2C scanner first)
```

```
#define LCD_I2C_ADDRESS 0x27
```

```
#define LCD_COLS      16
```

```
#define LCD_ROWS      2
```

```
LiquidCrystal_I2C lcd(LCD_I2C_ADDRESS, LCD_COLS, LCD_ROWS);
```

```
// ——— State Variables —————
```

```
float clutchPercent = 0.0f;
```

```
float engineRPM    = 0.0f;
```

```
float vehicleSpeed = 0.0f;
```

```
bool brakePressed = false;
```

```
bool isStall      = false;
```

```
bool isBrakeWarn  = false;
```

```
// ——— Buzzer Non-blocking State —————
```

```
bool buzzerActive = false;
```

```
bool buzzerState = false;    // current HIGH/LOW
```

```
unsigned long lastBuzzerToggle = 0;
```

```
// ——— LCD Refresh Throttle —————
```

```
unsigned long lastLcdUpdate = 0;
```

```
#define LCD_REFRESH_MS 150    // update LCD every 150ms
```

```
// ——— ADC Smoothing —————
```

```
#define ADC_SAMPLES 8        // average over N readings
```

```
int adcBuffer[3][ADC_SAMPLES]; // [channel][sample]
```

```
int adcIndex = 0;
```

```
// ——— Custom LCD Characters —————
```

```
// Small car icon for idle state (5×8 pixels)
```

```
byte carChar[8] = {
```

```
    0b00000,
```

```
    0b01110,
```

```
    0b11111,
```

```
    0b11111,
```

```
    0b01010,
```

```
    0b00000,
```

```
    0b00000,
```

```
    0b00000
```

```
};
```

```
// ——— Function Prototypes ———
void readSensors();
float readADCSmoothed(int channel, int pin);
void evaluateLogic();
void updateLCD();
void updateLEDs();
void updateBuzzer();
void printPadded(float val, int totalWidth, int decimals);
void clearLcdLine(int row);

// =====
void setup() {
  Serial.begin(115200);
  Serial.println("\n=== ESP32 Vehicle Stall Simulator ===");
  Serial.println("Initialising...");

  // Configure output pins
  pinMode(PIN_LED_RED, OUTPUT);
  pinMode(PIN_LED_GREEN, OUTPUT);
  pinMode(PIN_BUZZER, OUTPUT);

  // Configure input pins
  // GPIO34/35 are input-only; no INPUT_PULLUP available on them
  // GPIO32/33 support INPUT_PULLUP
  pinMode(PIN_POT_CLUTCH, INPUT);
  pinMode(PIN_POT_RPM, INPUT);
  pinMode(PIN_POT_SPEED, INPUT);
  pinMode(PIN_BTN_BRAKE, INPUT_PULLUP); // active LOW — external 10kΩ also fine

  // Set ADC resolution and attenuation for full 0–3.3V range
  analogReadResolution(12);           // 12-bit = 0–4095
  analogSetAttenuation(ADC_11db);     // 0–3.6V input range

  // Initialise ADC smoothing buffers
  for (int ch = 0; ch < 3; ch++)
    for (int s = 0; s < ADC_SAMPLES; s++)
      adcBuffer[ch][s] = 0;

  // Initialise I2C LCD
  Wire.begin(21, 22);                 // SDA, SCL
```

```

lcd.init();
lcd.backlight();
lcd.createChar(0, carChar);

// Splash screen
lcd.setCursor(0, 0); lcd.print(" Stall Simulator");
lcd.setCursor(0, 1); lcd.print(" Initialising..");
delay(1500);
lcd.clear();

// Ensure all outputs are OFF at start
digitalWrite(PIN_LED_RED, LOW);
digitalWrite(PIN_LED_GREEN, HIGH); // green ON = normal at startup
digitalWrite(PIN_BUZZER, LOW);

Serial.println("Ready.");
}

// =====
void loop() {
  readSensors();
  evaluateLogic();
  updateLEDs();
  updateBuzzer();

  // Throttle LCD updates to avoid flicker
  if (millis() - lastLcdUpdate >= LCD_REFRESH_MS) {
    updateLCD();
    lastLcdUpdate = millis();
  }

  // Serial debug output (useful during testing)
  Serial.printf("CLT:%.1f%% RPM:%.0f SPD:%.1f km/h BRK:%s STALL:%s BRAKE_WARN:%s\n",
    clutchPercent, engineRPM, vehicleSpeed,
    brakePressed ? "ON" : "off",
    isStall ? "YES" : "no",
    isBrakeWarn ? "YES" : "no"
  );

  delay(20); // 50 Hz main loop
}

```

```

// —— Read and smooth all sensor inputs ——
void readSensors() {
    // Read raw ADC into rolling buffers
    adcBuffer[0][adcIndex] = analogRead(PIN_POT_CLUTCH);
    adcBuffer[1][adcIndex] = analogRead(PIN_POT_RPM);
    adcBuffer[2][adcIndex] = analogRead(PIN_POT_SPEED);
    adcIndex = (adcIndex + 1) % ADC_SAMPLES;

    // Average each channel
    long sum0 = 0, sum1 = 0, sum2 = 0;
    for (int i = 0; i < ADC_SAMPLES; i++) {
        sum0 += adcBuffer[0][i];
        sum1 += adcBuffer[1][i];
        sum2 += adcBuffer[2][i];
    }
    float avg0 = (float)sum0 / ADC_SAMPLES;
    float avg1 = (float)sum1 / ADC_SAMPLES;
    float avg2 = (float)sum2 / ADC_SAMPLES;

    // Map ADC values to physical units
    clutchPercent = (avg0 / ADC_MAX) * CLUTCH_MAX;
    engineRPM    = (avg1 / ADC_MAX) * RPM_MAX;
    vehicleSpeed = (avg2 / ADC_MAX) * SPEED_MAX;

    // Brake button: LOW = pressed (internal/external pull-up)
    brakePressed = (digitalRead(PIN_BTN_BRAKE) == LOW);
}

// —— Evaluate stall and brake warning logic ——
void evaluateLogic() {
    /*
    * STALL condition:
    * clutch > 10% (pedal NOT fully depressed)
    * AND (RPM < 1000 OR speed > 5 km/h)
    * Real-world: releasing clutch too fast at low RPM, or
    * engaging clutch while still rolling without enough revs.
    */
    isStall = (clutchPercent > STALL_CLUTCH_THRESHOLD) &&
        (engineRPM < STALL_RPM_THRESHOLD || vehicleSpeed > STALL_SPEED_THRESHOLD);
}

```

```

/*
 * BRAKE WARNING condition:
 * Brake pedal pressed AND speed > 50 km/h
 * Simulates hard braking at highway speed.
 */
isBrakeWarn = brakePressed && (vehicleSpeed > BRAKE_SPEED_THRESHOLD);
}

// ——— Update LCD display —————
/*
 * LCD Layout:
 * Row 0: "C:XXX% R:XXXXRPM" (clutch %, RPM)
 * Row 1: "S:XXXkm/h [STATUS]" (speed, status flag)
 *
 * STATUS field cycles: "OK" | "STALL" | "BRAKE!"
 * Brake status shown inline when pressed.
 */
void updateLCD() {
    // — Row 0: Clutch + RPM —
    lcd.setCursor(0, 0);
    lcd.print("C:");
    // Clutch: 3 digits, no decimal (e.g. "100", " 45", " 7")
    if (clutchPercent < 10.0f)    lcd.print(" ");
    else if (clutchPercent < 100.0f) lcd.print(" ");
    lcd.print((int)clutchPercent);
    lcd.print("% ");

    lcd.print("R:");
    // RPM: 4 digits
    int rpm = (int)engineRPM;
    if (rpm < 10)    lcd.print(" ");
    else if (rpm < 100) lcd.print(" ");
    else if (rpm < 1000) lcd.print(" ");
    lcd.print(rpm);

    // — Row 1: Speed + Status —
    lcd.setCursor(0, 1);
    lcd.print("S:");
    int spd = (int)vehicleSpeed;
    if (spd < 10)    lcd.print(" ");
    else if (spd < 100) lcd.print(" ");

```

```

lcd.print(spd);
lcd.print("km/h ");

// Status field (right side of row 1)
if (isStall) {
    lcd.print("STALL ");
} else if (isBrakeWarn) {
    lcd.print("BRAKE!");
} else if (brakePressed) {
    lcd.print("BRK  ");
} else {
    lcd.print(" OK ");
}
}

// ——— Update LEDs —————
void updateLEDs() {
    if (isStall || isBrakeWarn) {
        // Warning state: RED on, GREEN off
        digitalWrite(PIN_LED_RED, HIGH);
        digitalWrite(PIN_LED_GREEN, LOW);
    } else {
        // Normal state: RED off, GREEN on
        digitalWrite(PIN_LED_RED, LOW);
        digitalWrite(PIN_LED_GREEN, HIGH);
    }
}

// ——— Non-blocking buzzer beep pattern —————
/*
 * Uses millis()-based toggling instead of delay() so the
 * main loop stays responsive during buzz patterns.
 *
 * Stall pattern: fast double beep (150ms on / 150ms off)
 * Brake pattern: faster rapid beep (80ms on / 80ms off)
 * No warning:  buzzer OFF
 */
void updateBuzzer() {
    unsigned long now = millis();

    if (!isStall && !isBrakeWarn) {

```

```

// No warning — ensure buzzer is silent
digitalWrite(PIN_BUZZER, LOW);
buzzerState = false;
return;
}

// Choose timing based on active warning
unsigned long onTime = isStall ? BUZZER_BEEP_ON_MS : BUZZER_BRAKE_ON_MS;
unsigned long offTime = isStall ? BUZZER_BEEP_OFF_MS : BUZZER_BRAKE_OFF_MS;
unsigned long interval = buzzerState ? onTime : offTime;

if (now - lastBuzzerToggle >= interval) {
  buzzerState = !buzzerState;
  digitalWrite(PIN_BUZZER, buzzerState ? HIGH : LOW);
  lastBuzzerToggle = now;
}
}

// ——— Helper: I2C address scanner (call from setup for debug) —
void scanI2C() {
  Serial.println("Scanning I2C bus...");
  for (byte addr = 1; addr < 127; addr++) {
    Wire.beginTransmission(addr);
    if (Wire.endTransmission() == 0) {
      Serial.printf(" Found device at 0x%02X\n", addr);
    }
  }
  Serial.println("Scan complete.");
}

```

VI. Conclusion and Future Work

A functional, low-cost ESP32-based vehicle stalling simulator has been designed, implemented, and validated. The system identifies stall-prone clutch–RPM–speed combinations and brake-overspeed events in real time, providing feedback through an LCD, LEDs, and a buzzer. All five test cases passed in simulation, supporting the correctness of the rule-based detection logic. The total bill of materials is less than US\$15, making the platform accessible to schools and vocational training centres with limited budgets.

Future work will extend the platform in three directions. First, a stepper-motor actuated steering feedback mechanism will introduce haptic alerts for imminent stall conditions. Second, the rule engine will be replaced by a lightweight machine-learning classifier trained on data recorded from real vehicles to improve stall prediction accuracy. Third, Wi-Fi telemetry will be added to stream session data to a cloud dashboard, enabling instructors to review learner performance remotely.

References

- [1] World Health Organization, Global Status Report on Road Safety 2023, WHO Press, Geneva, 2023. [Online]. Available: <https://www.who.int/publications/i/item/9789240086517>
- [2] Espressif Systems, ESP32 Technical Reference Manual v5.2, Espressif Systems, Shanghai, 2024. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [3] D. Vallejo-Huanga, P. Morillo, and C. Porras, "Low-Cost Embedded Driving Simulator Using Arduino and Open-Source Tools," in *Proc. IEEE Int. Conf. Inf. Syst. Comput. Sci. (INCISCOS)*, Quito, Ecuador, 2019, pp. 234–240.
- [4] P. Blum, LiquidCrystal_I2C Arduino Library, GitHub Repository, 2021. [Online]. Available: https://github.com/johnrickman/LiquidCrystal_I2C
- [5] U. Banzi and M. Shiloh, *Getting Started with Arduino*, 3rd ed., Maker Media, Sebastopol, CA, 2014, pp. 55–78.